Mendel University in Brno
Faculty of Business and Economy

Sentiment Classification, Topic Clustering and Similarity Search in Hotel Reviews

Semester project

Bc. Jan Dub
Bc. David Krčmář
Bc. René Mrákava

Supervisor:
doc. Ing. František Dařena, Ph.D.

Brno 2026

Abstract

Dub, J.; Krčmář, D.; Mrákava, R. Sentiment Classification, Topic Clustering and Similarity Search in Hotel Reviews. Semester project. Mendel University in Brno, 2026.

This semester project presents a compact end-to-end text-mining pipeline for hotel reviews from the Kaggle dataset *515K Hotel Reviews Data in Europe*. The work validates and summarizes the original CSV file, preprocesses review text, trains and compares three classical classifiers, evaluates chi-square feature selection, clusters negative reviews with K-Means, and builds a TF-IDF retrieval module with cosine similarity. The dataset contains 515,738 reviews and 17 columns. In the final benchmark run, sentiment classification used the full available dataset with **TF-IDF unigrams and bigrams, `SelectKBest(chi2, k=20000)` and Linear SVM**, reaching **macro F1 = 0.8510** and **accuracy = 95.90%**. Negative-review clustering on a 50,000-row sample remains interpretable but weakly separated, with **k = 7** and **silhouette score = 0.0085**. Retrieval works well for specific multi-word queries and degrades for generic queries. The report discusses both the practical usefulness and the limits of classical text-mining methods on noisy user-generated review data.

Key words: text mining, hotel reviews, sentiment classification, clustering, information retrieval, TF-IDF, Linear SVM

Contents

Glossary

- ***TF-IDF***: a weighted text representation that highlights terms important for a specific document while down-weighting terms that are frequent across the whole collection.
- ***n-gram***: a sequence of n tokens; this project compares unigrams with a combination of unigrams and bigrams.
- ***Linear SVM***: a linear Support Vector Machine classifier suitable for sparse text representations with many features.
- ***Chi-square feature selection***: a feature-selection method based on the statistical dependence between a term and the target class.
- ***K-Means***: a clustering algorithm that partitions documents into a predefined number of clusters according to their distance from cluster centroids.
- ***Silhouette score***: an internal clustering metric that evaluates how compact a cluster is and how well it is separated from other clusters.
- ***Cosine similarity***: a vector similarity measure based on the angle between two vectors; it is a standard choice for comparing TF-IDF representations.

1 Introduction and motivation

Hotel reviews are a suitable example of real-world text data on which the main tasks of text mining can be demonstrated within one shared pipeline. Each review combines free text, a numerical score, and additional metadata. This combination makes it possible to address sentiment classification, topic clustering, and similar-document search at the same time.

The practical motivation is twofold. First, hotel reviews are a commercially important source of feedback for accommodation providers and booking platforms such as Booking.com. Second, the dataset contains realistic noise: placeholders such as `No Negative`, very short entries, variable review length, and an imbalanced distribution of target classes. The project is therefore useful not only for demonstrating working methods, but also for discussing their limits openly.

From the perspective of the course, the work covers text representation, preprocessing, classification, feature selection, clustering, and information retrieval, which are standard tasks in text mining and NLP [4, 5]. Instead of focusing on one isolated task, the project creates a consistent text-mining workflow with a shared dataset and a common evaluation logic.

2 Project goal and research questions

The goal of this project is to design and implement a compact text-mining pipeline for hotel-review analysis and to evaluate which classical methods work best on this type of data.

2.1 Main objectives

1. prepare and document a hotel-review dataset,
2. create a shared text representation suitable for machine learning,
3. train and compare several sentiment-classification models,
4. evaluate the effect of feature selection using `SelectKBest(chi2)`,
5. identify the main themes in negative reviews using clustering,
6. create a simple retrieval module returning similar reviews for a given query.

2.2 Research questions

1. Which of the three classical models (`MultinomialNB`, `LogisticRegression`, `LinearSVC`) achieves the best performance on hotel reviews?
2. Does feature selection using `SelectKBest(chi2)` improve performance compared with a plain TF-IDF baseline?
3. Which themes dominate negative reviews, and how interpretable are the resulting clusters?
4. How well does TF-IDF and cosine-similarity retrieval work for specific and generic queries?

3 Dataset

3.1 Data source and structure

The primary data source is the Kaggle dataset *515K Hotel Reviews Data in Europe* by Jiashen Liu [1]. The project uses the original file `Hotel_Reviews.csv`, which is expected in the local project structure under `data/raw/`. The dataset contains the text fields `Positive_Review` and `Negative_Review`, the numerical score `Reviewer_Score`, and additional metadata such as `Hotel_Name`, `Average_Score`, `Tags`, and the hotel location.

Table 1: Basic dataset characteristics

Item	**Value**
Source	Kaggle: *515K Hotel Reviews Data in Europe*
Main file	`Hotel_Reviews.csv`
Number of rows	515,738
Number of columns	17
Key attributes	`Positive_Review`, `Negative_Review`, `Reviewer_Score`, `Hotel_Name`, `Tags`
Missing values	0 in the review-text fields and score; 3,268 missing values in `lat`/`lng`
`Reviewer_Score` range	2.5 to 10.0
Mean / median score	8.3951 / 8.8

3.2 Data quality and practical limitations

The dataset is large, but it is not clean. Many reviews use placeholders such as `No Negative` or `No Positive`. In the verified local snapshot, there are 152,663 values of `No Negative` and 37,791 values of `No Positive`. In addition, short pseudo-negative expressions such as `No complaints`, `Everything was perfect`, or simply `Breakfast` occur in the data. These are not technical placeholders, but they complicate further processing.

Another limitation is the strongly positive score distribution. The most frequent value in the full dataset is 10.0, appearing 115,853 times, while the lower part of the scale is much less common. This is important for label design and for interpreting accuracy, because accuracy alone is not sufficient for an imbalanced classification problem.

3.3 Review-score distribution

The distribution of `Reviewer_Score` is shown in Figure 1. The high concentration of scores between 8.3 and 10.0 confirms that the dataset has a natural positive bias.

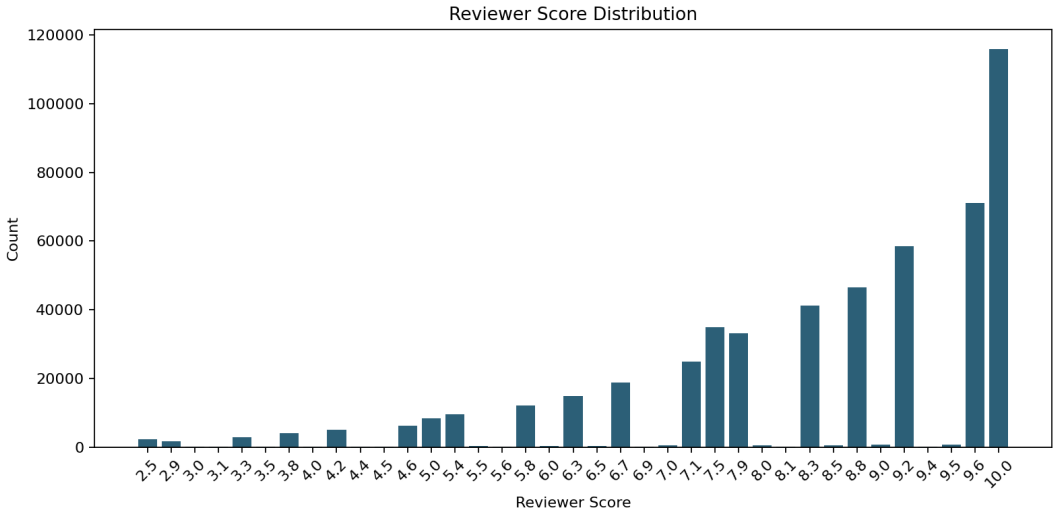

Figure 1: Distribution of `Reviewer_Score` in the full dataset

3.4 Working mode

The project uses different dataset scopes for different tasks. Classification and the final retrieval query were run on the full available dataset without downsampling. Clustering remains sample-based with 50,000 records, because full-dataset clustering is slower, harder to interpret, and does not automatically improve cluster separation. This compromise fits the educational nature of the project: the main goal is to compare methods and interpret results, not to maximize model performance at the cost of long runtime and higher memory requirements.

3.5 Reproducibility note

The dataset is not included in the repository because of its size and distribution through Kaggle. After downloading `Hotel_Reviews.csv` into `data/raw/`, the final benchmark and the supporting report figures can be reproduced with the following commands:

```
python src/classification.py --data data/raw/Hotel_Reviews.csv --full-dataset --use-
    bigrams --feature-selection-k 20000
python src/feature_selection_experiments.py --data data/raw/Hotel_Reviews.csv --sample-
    size 50000 --use-bigrams
python src/clustering.py --data data/raw/Hotel_Reviews.csv --sample-size 50000 --k 7 --
    use-bigrams
python src/clustering_experiments.py --data data/raw/Hotel_Reviews.csv --sample-size
    50000 --k-values 5,6,7,8,9 --use-bigrams --output-dir outputs/
    clustering_50k_comparison
python src/retrieval.py --data data/raw/Hotel_Reviews.csv --full-dataset --query "dirty␣
    room␣and␣noisy␣street" --top-k 5
python src/retrieval_experiments.py --data data/raw/Hotel_Reviews.csv --sample-size
    20000 --use-bigrams
python src/generate_report_figures.py --data data/raw/Hotel_Reviews.csv --clustering-
    comparison outputs/clustering_50k_comparison/clustering_k_comparison.csv
```

4 Data preprocessing

4.1 Text-attribute construction

The project creates three shared text variants:

- `review_text`: concatenation of the positive and negative parts of the review,
- `negative_text`: only the negative part of the review,
- `positive_text`: only the positive part of the review.

For sentiment classification, `review_text` proved to be the best option because it combines both sides of the review and gives the model the most context. For clustering, `negative_text` was used because problem topics are more readable in that field.

4.2 Cleaning pipeline

Before text representation, the following steps were applied:

1. replace placeholders in `Positive_Review` and `Negative_Review` with empty text,
2. remove records with empty selected text fields,
3. convert text to lowercase,
4. remove non-alphabetic characters,
5. normalize whitespace,
6. remove records that remained empty after cleaning.

On the full available dataset, binary label filtering and preprocessing produced the following data flow:

Table 2: Data flow after labeling and preprocessing

Step	**Number of rows**
Loaded dataset	515,738
After binary labeling	366,349
After filtering empty text fields	365,997
After final cleaning	365,988
Training split	292,790
Test split	73,198

4.3 Text representation and labels

The final classification baseline uses the TF-IDF representation from scikit-learn [2]. TF-IDF is a standard sparse vector-space representation for text retrieval and related text-mining tasks [3]. Both unigrams and a combination of unigrams and bigrams were tested; the second option performed better. The selected configuration was:

- `ngram_range = (1, 2)`,
- `min_df = 5`,
- `max_df = 0.9`,
- `max_features = 50000`,
- the English stop-word list from scikit-learn.

Sentiment labels were created as binary labels:

- `Reviewer_Score <= 5.0` → negative,
- `Reviewer_Score >= 8.0` → positive,
- the middle score range was excluded from classification.

This design intentionally increases class separability, but it also reduces the number of usable records. After filtering, the full benchmark dataset contained 335,365 positive and 30,623 negative reviews, confirming strong class imbalance.

5 Sentiment classification

5.1 Models and evaluation setup

Three classical models suitable for sparse text representations were compared. The implementation uses scikit-learn [2]; Linear SVM follows the margin-based classification principle introduced by support vector machines [6].

- `Multinomial Naive Bayes`,
- `Logistic Regression`,
- `Linear SVM`.

The data was split into stratified training and test sets in an 80/20 ratio. Because of class imbalance, accuracy is reported together with macro precision, macro recall, and macro F1. This is important because high accuracy could otherwise hide weak performance on the minority negative class.

5.2 Final benchmark results with feature selection

Table 3 summarizes the final benchmark results on `review_text` using TF-IDF unigrams and bigrams with `SelectKBest(chi2, k=20000)`.

Table 3: Final sentiment-classification results with `SelectKBest(chi2, k=20000)`

Model	**Macro prec.**	**Macro rec.**	**Macro F1**	**Acc.**
Linear SVM	0.8977	0.8158	0.8510	0.9590
Logistic Regression	0.9131	0.7938	0.8412	0.9582
Multinomial Naive Bayes	0.9044	0.7726	0.8230	0.9544

Linear SVM ranked first in the final benchmark. Logistic Regression remained competitive, but it lagged mainly in macro recall. Naive Bayes achieved acceptable accuracy, but it performed worse on the minority negative class, which reduced macro F1.

5.3 Precision-Recall and ROC comparison

Because the negative class is much smaller than the positive class, the Precision-Recall curve in Figure 2 uses *negative review* as the positive event. This makes the plot directly focused on the practically important task: finding dissatisfied customers. Logistic Regression has the highest ranking-based average precision (AP = 0.812), while Linear SVM has the best thresholded macro F1 in Table 3. The difference shows that ranking quality and the final hard-label decision are related, but not identical, evaluation views.

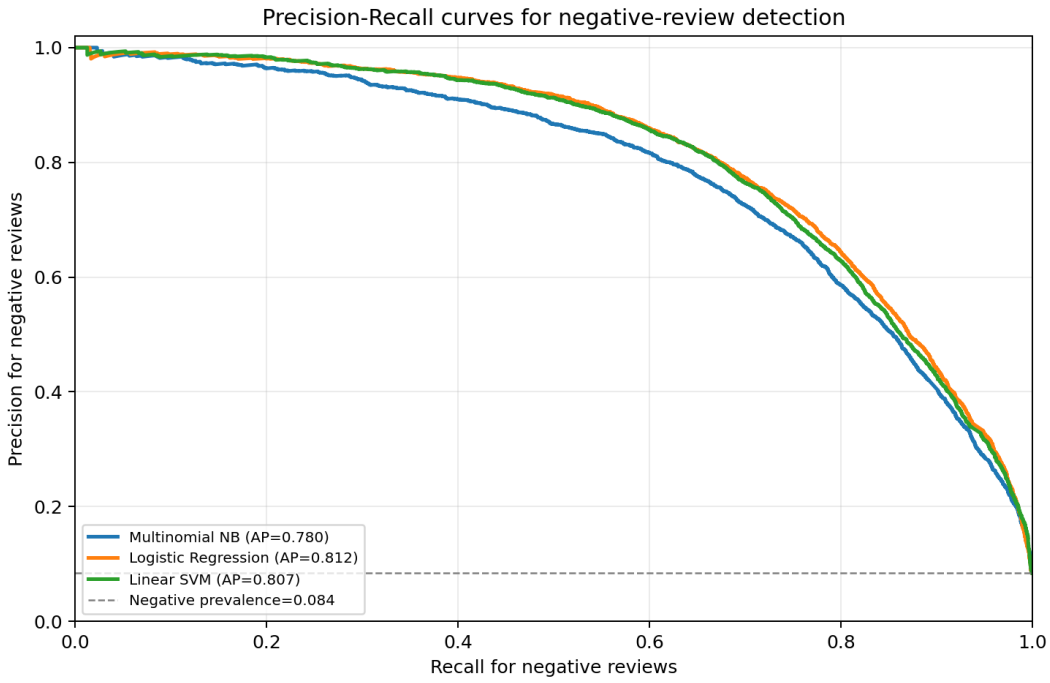

Figure 2: Precision-Recall curves for negative-review detection

The ROC curves in Figure 3 are included as a supplementary view. All three models achieve high ROC-AUC values above 0.95, but ROC-AUC is less sensitive to the strong class imbalance than the Precision-Recall view.

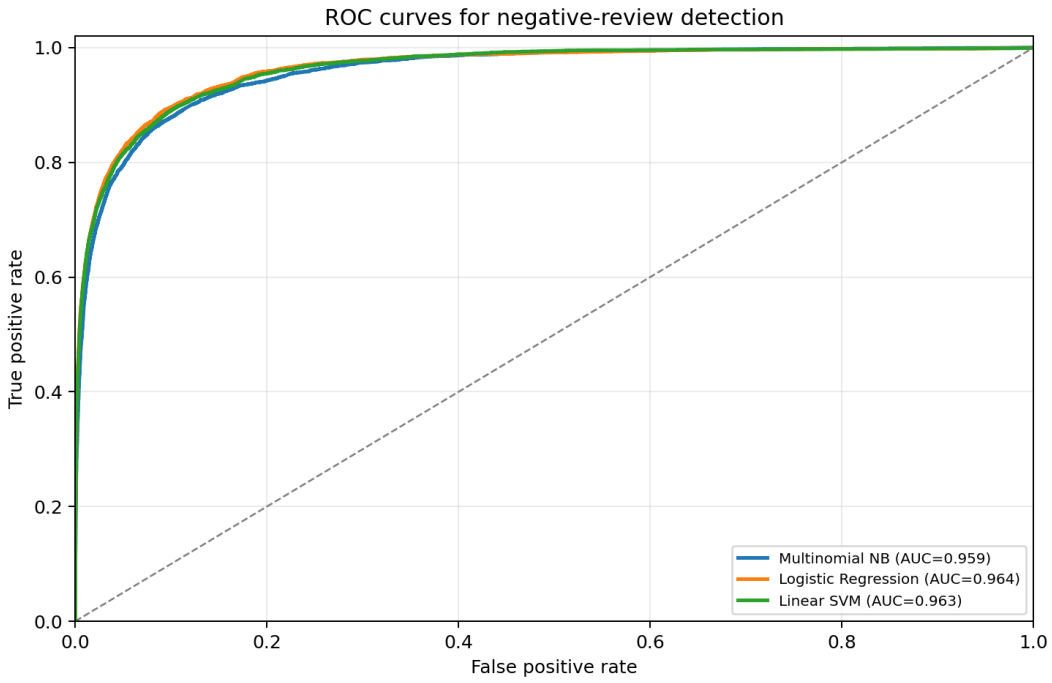

Figure 3: ROC curves for negative-review detection

5.4 Interpretation of the best final model

For the best final model (`Linear SVM`), the confusion matrix shows the following behavior:

Table 4: Confusion matrix for final Linear SVM

Actual / predicted class	**Negative**	**Positive**
Negative	3,944 (64.39%)	2,181 (35.61%)
Positive	823 (1.23%)	66,250 (98.77%)

The model recognizes positive reviews very well, but it still assigns part of the negative reviews to the positive class. Out of 6,125 negative test reviews, it correctly identifies 3,944 reviews (64.39%) and incorrectly marks 2,181 reviews (35.61%) as positive. These percentages are the most important part of the confusion matrix, because negative reviews incorrectly predicted as positive are false negatives for negative-review detection: problematic stays are missed. This behavior corresponds to the nature of the data: the negative class is smaller, lexically less homogeneous, and more likely to contain short or mixed formulations.

6 Feature selection

6.1 Method and experimental setup

In the next phase, `SelectKBest(chi2)` from scikit-learn was applied to the same TF-IDF representation [2]. The goal was to test whether feature selection can remove part of the noise, improve generalization, and provide more interpretable terms for sentiment analysis.

Feature selection was first compared on a 50,000-row development sample with $k \in \{0, 5000, 10000, 20000, 30000\}$, where $k = 0$ means the baseline without feature selection. For that smaller experiment, the actual number of active features after vectorization was only 15,266; therefore, the configurations with $k = 20000$ and $k = 30000$ effectively returned all attributes and reproduced the baseline.

6.2 Configuration comparison

Table 5: Effect of feature selection on the best model (Linear SVM)

k	Macro F1	Accuracy
0	0.8039	0.9481
5,000	0.8120	0.9503
10,000	0.8032	0.9479
20,000	0.8039	0.9481
30,000	0.8039	0.9481

The same comparison is visualized in Figure 4. The best development-sample result was obtained with `k = 5000`. For the final full-dataset benchmark, the project used `k = 20000` to retain more potentially useful terms in the much larger corpus while still applying chi-square feature selection. The graph also shows that the differences are relatively small: feature selection helps, but it is not the main source of the classification performance.

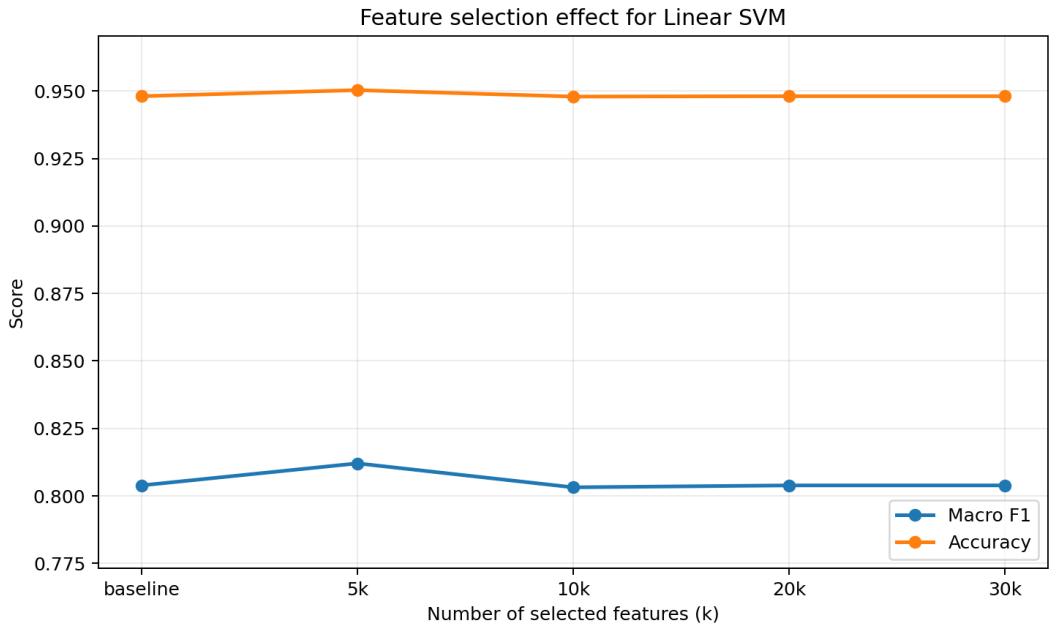

Figure 4: Comparison of the Linear SVM baseline and selected values of k

6.3 Most important sentiment terms

Feature selection also supports interpretation. For the final configuration (`Linear SVM + k = 20000`), the strongest indicators included the following terms:

Table 6: Selected sentiment terms

Positive sentiment	**Negative sentiment**
excellent	dirty
amazing	worst
lovely	disinterested
fantastic	filthy
great	unclean
loved	impolite
perfect	rude
definitely stay	unhelpful

The terms are semantically interpretable and confirm that classical linear models over TF-IDF provide a meaningful signal for this type of data.

7 Clustering negative reviews

7.1 Input data and method

Clustering was performed on `negative_text`, because this field is the most useful for identifying repeated problem types. From a 50,000-row sample, 35,088 documents remained after removing empty and uninformative texts. TF-IDF with unigrams and bigrams was used, followed by K-Means from scikit-learn [2]. K-Means is a standard partitional clustering method [7].

7.2 Choosing the number of clusters

Several values of k were compared on the same 50,000-row clustering sample. The comparison is summarized in Table 7 and visualized in Figure 5.

Table 7: K-Means comparison for different values of k

k	Silhouette score	Min. cluster size	Max. cluster size
5	0.0070	1,087	27,447
6	0.0079	1,614	16,839
7	0.0085	366	19,397
8	0.0114	152	20,100
9	0.0082	309	21,553

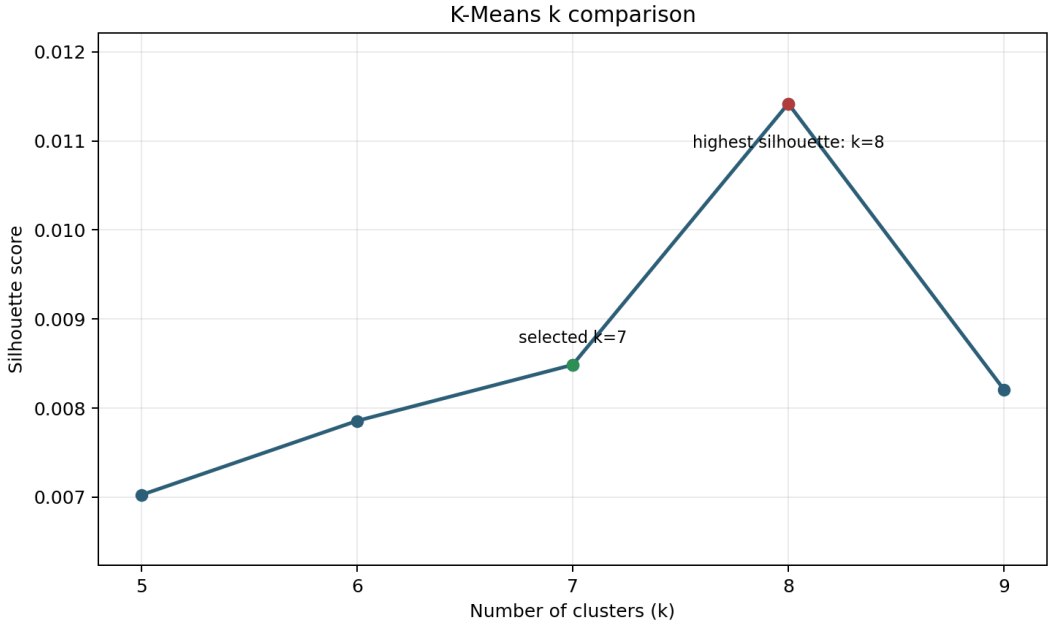

Figure 5: Silhouette-score comparison for K-Means clustering

The silhouette score is an internal measure of cluster compactness and separation [8]. All tested values are very close to zero, so the result must be interpreted carefully: the clusters are readable, but they are not sharply separated. The highest silhouette score in this run was obtained for $k = 8$, but that configuration created a very small pseudo-negative cluster dominated by terms such as `perfect` and `just perfect`, while still keeping one very large mixed cluster. The final choice $k = 7$ is therefore a practical compromise: it keeps the number of clusters simple, avoids an extra tiny artificial cluster, and still separates several useful hotel-problem themes such as Wi-Fi/air conditioning, small rooms, breakfast, and parking.

7.3 Interpretation of the final clusters

For the selected final configuration, the clusters were manually named according to top terms and representative reviews:

Table 8: Manual interpretation of clusters for $k = 7$

Cluster	Size	Interpretation
0	366	generic dislike or short negative fragments
1	8,282	room comfort, service, bathroom, and night issues
2	1,435	Wi-Fi and air-conditioning problems
3	19,397	broad miscellaneous hotel and staff comments
4	2,016	small-room complaints
5	2,980	breakfast quality, price, and inclusion
6	612	parking and car-access costs

The cleanest clusters are those focused on Wi-Fi, air conditioning, small rooms, breakfast, and parking. The weakest result is the largest cluster, which captures a mixture of broad hotel, staff, room, and bathroom comments.

7.4 Weaknesses of clustering

The low silhouette score and the existence of a large "miscellaneous" cluster are directly related to input-text quality. The data still contains short or pseudo-negative formulations such as `No failings`, `No complaints`, `Everything was perfect`, `Breakfast`, or `Expensive`. These expressions reduce thematic purity and distort centroids. Therefore, clustering should be understood mainly as an exploratory tool for summarizing frequent problems, not as a final customer segmentation.

8 Information retrieval

8.1 Retrieval-module design

The retrieval module uses the same vector-space principle as the classification representation, but it is applied to `review_text`. In the final run, retrieval was configured for the full available dataset without downsampling. The TF-IDF representation uses up to 30,000 unigram features, and retrieval is performed using cosine similarity, a common similarity measure in vector-space information retrieval [3].

8.2 Example queries

The final reproducibility run used the query `dirty room and noisy street`. A separate multi-query development check also covered several specific and generic formulations:

- `dirty room and noisy street`,
- `friendly staff and great location`,
- `small bathroom and old furniture`,
- `excellent breakfast and comfortable bed`,
- `slow wifi and poor service`,
- `good hotel`.

8.3 Similarity-score based assessment

Table 9: Retrieval-query comparison using cosine similarity

Query	**Top cosine**	**Mean top-5**	**Interpretation note**
friendly staff and great location	1.0000	0.8233	very close lexical matches
small bathroom and old furniture	0.6613	0.5112	concrete negative aspects
excellent breakfast and comfortable bed	0.5636	0.5301	two clear positive aspects
dirty room and noisy street	0.5671	0.5184	noise captured strongly
slow wifi and poor service	0.6049	0.4904	Wi-Fi signal dominates
good hotel	1.0000	0.8592	generic query, low specificity

The same scores are shown in Figure 6. The cosine values should not be interpreted as manual relevance labels. They measure lexical similarity in TF-IDF space. This is why the generic query `good hotel` receives high cosine scores, even though it is less useful for analysis than a more specific query. Retrieval works best when the query contains concrete aspects of the hotel experience. In the final full-dataset run, the top results for `dirty room and noisy street` were short but relevant, including `very noisy dirty`, `room on the street side very noisy`, and `the room was very noisy from the street`. For generic queries, the model naturally prefers short and frequent positive phrases, which reduces the usefulness of the results.

Figure 6: Top and mean top-5 cosine similarity for retrieval queries

9 Discussion

9.1 What worked best

The most stable part of the pipeline was sentiment classification. In the final full-dataset benchmark, Linear SVM with TF-IDF and chi-square feature selection reached macro F1 = 0.8510 and accuracy = 95.90%. This confirms that linear models over sparse representations remain strong and interpretable for this type of data.

The retrieval module also performed well. For specific multi-word queries, it returned meaningful and easily interpretable results without requiring more complex embedding models.

9.2 What was problematic

The main problem in the dataset is noise. In addition to explicit placeholders, the data contains short, semantically ambiguous, or only formally negative entries. This noise directly affects clustering, where it leads to weak separation and to one dominant miscellaneous cluster.

The second problem is class imbalance after binary labeling. Classification accuracy is very high, but without macro metrics it would easily hide the fact that the model still has substantially weaker recall on the negative class than on the positive class.

9.3 Limitations of the work

- clustering was still run on a 50,000-row sample rather than on the full dataset,
- retrieval was evaluated with cosine scores and manual inspection, without a manually annotated relevance benchmark,
- clustering was interpreted manually and without external ground truth,
- preprocessing uses simple cleaning without lemmatization, stemming, or transformer embeddings.

9.4 Future work

Natural extensions of the project include:

- expanding the placeholder dictionary and filtering short pseudo-negative phrases more precisely,
- running additional full-dataset feature-selection comparisons,
- comparing K-Means with NMF or topic modeling,

- extending retrieval with BM25 or sentence embeddings,
- trying class weighting or targeted sampling to improve recall for the negative class.

10 Conclusion

The project showed that classical text-mining methods can produce solid and interpretable results on hotel reviews. After validating and cleaning the dataset, a unified pipeline was created for text representation, sentiment classification, feature selection, clustering, and retrieval.

The best final classification configuration uses `review_text`, TF-IDF with unigrams and bigrams, `SelectKBest(chi2, k=20000)`, and Linear SVM. This configuration achieved macro F1 = 0.8510 and accuracy = 95.90%. Clustering of negative reviews produced useful thematic summaries, especially for Wi-Fi, air conditioning, small rooms, breakfast, and parking, but the internal separation of clusters was weak. The retrieval module confirmed that TF-IDF and cosine similarity remain practically usable for specific queries.

Overall, the project met its intended goal: it demonstrated a complete text-mining workflow on one realistic dataset, including both strengths and limitations. The results are concrete enough for the written report and for project defense, while also leaving clear space for further extensions.

11 References

1. Liu, J. *515K Hotel Reviews Data in Europe*. Kaggle dataset. Available from: `https://www.kaggle.com`. Dataset path: `datasets/jiashenliu/515k-hotel-reviews-data-in-europe`.

2. Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2011, pp. 2825–2830.

3. Manning, C. D.; Raghavan, P.; Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

4. Žižka, J.; Dařena, F.; Svoboda, A. *Text Mining with Machine Learning: Principles and Techniques*. CRC Press, 2019.

5. Jurafsky, D.; Martin, J. H. *Speech and Language Processing*. 3rd ed. draft, 2026. Available from: `https://web.stanford.edu/~jurafsky/slp3/`.

6. Cortes, C.; Vapnik, V. Support-vector networks. *Machine Learning*, 20, 1995, pp. 273–297.

7. MacQueen, J. Some methods for classification and analysis of multivariate observations. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1967, pp. 281–297.

8. Rousseeuw, P. J. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 1987, pp. 53–65.